2 Conditions

The `where` method allows you to specify conditions to limit the records returned, representing the `WHERE`-part of the SQL statement. Conditions can either be specified as a string, array, or hash.

2.1 Pure String Conditions

If you'd like to add conditions to your find, you could just specify them in there, just like `Client.where("orders_count = '2'")`. This will find all clients where the `orders_count` field's value is 2.
Building your own conditions as pure strings can leave you vulnerable to SQL injection exploits. For example, `Client.where("first_name LIKE '%#{params[:first_name]}%'")` is not safe. See the next section for the preferred way to handle conditions using an array.

2.2 Array Conditions

Now what if that number could vary, say as an argument from somewhere? The find would then take the form:

```
Client.where("orders_count = ?", params[:orders])
```

Active Record will go through the first element in the conditions value and any additional elements will replace the question marks `(?)` in the first element.
If you want to specify multiple conditions:

```
Client.where("orders_count = ? AND locked = ?", params[:orders],
false)
```

In this example, the first question mark will be replaced with the value in `params[:orders]` and the second will be replaced with the SQL representation of `false`, which depends on the adapter.
This code is highly preferable:

```
Client.where("orders_count = ?", params[:orders])
```

to this code:

```
Client.where("orders_count = #{params[:orders]}")
```

because of argument safety. Putting the variable directly into the conditions string will pass the variable to the database **as-is**. This means that it will be an unescaped variable directly from a user who may have malicious intent. If you do this, you put your entire database at risk because once a user finds out they can exploit your database they can do just about anything to it. Never ever put your arguments directly inside the conditions string.
For more information on the dangers of SQL injection, see the Ruby on Rails Security Guide.

2.2.1 Placeholder Conditions

Similar to the `(?)` replacement style of params, you can also specify keys/values hash in your array conditions:

```
Client.where("created_at >= :start_date AND created_at <= :end_date",
  {start_date: params[:start_date], end_date: params[:end_date]})
```

This makes for clearer readability if you have a large number of variable conditions.